SD · benchmark sortari

9 algoritmi.
16 teste.

Cine pierde, cine castiga, si cine pica.

N small = **20.000** · N big = **10.000.000** · timeout = **60s**

Cum am testat

Setup

Generator	**mt19937, seed 69420**
Compilator	`g++ -O2 -static -std=c++20`
Cronometru	`System.Diagnostics.Stopwatch`
Limita	**60 secunde / rulare**
N small	**20.000**
N big	**10.000.000**
Validare	`assert(is_sorted(...))`

8 tipuri de input

`random`	valori uniforme din mt19937
`sortat`	1, 2, 3, ..., n
`sortat_invers`	n, n-1, ..., 1
`few_unique`	doar 10 valori distincte
`almost_sorted`	sortat + 2% swap-uri aleatoare
`all_equal`	n copii ale lui 243
`neg_poz`	+/- amestecate uniform
`sawtooth`	1..n/2, apoi n/2..1

Cei 9 algoritmi

bubblesort — $O(n^2)$

Doi for-uri imbricate. Punct.

gnome_sort — $O(n^2)$

Pasi inainte cand e ok, inapoi cand nu.

insertionsort — $O(n^2)$

Imping cheia la pozitia ei. Rege pe sortat.

patiencesort — $O(n^2)$*

Pile + merge. Pentru noi: $O(n^2)$ in cel mai rau caz.

heap_sort — $O(n \log n)$

Heap binar, push_down recursiv.

merge_sort — $O(n \log n)$

Top-down + skip cand bucatile-s deja sortate.

quicksort — $O(n \log n)$

Pivot mijloc. Cade pe sawtooth.

introsort — $O(n \log n)$

Quick → heap la adancime mare → insertion sub 16.

radix_sort — $O(n \cdot k)$

LSD pe 8 biti × 4 treceri. XOR pe MSB pentru semn.

Rezultate small

Toti cei 9 algoritmi. Timp in ms. Galben = castigator pe coloana.

algoritm	random	sortat	sortat_invers	few_unique	almost_sorted	all_equal	neg_poz	sawtooth
bubblesort	656.3	126.9	309.0	134.7	218.3	120.3	561.9	160.0
gnome_sort	304.7	4.2	600.1	286.5	20.3	4.5	310.3	296.0
insertionsort	74.1	**3.1**	148.2	75.5	7.8	**3.2**	76.5	76.4
patiencesort	TIMEOUT	1608	5.6	147.9	1426	1523	26.1	633.9
heap_sort	5.4	5.1	5.1	5.9	4.9	3.9	**6.6**	6.1
merge_sort	8.4	4.4	**4.5**	5.5	4.6	3.5	8.0	**5.4**
quicksort	8.8	9.0	18.5	5.5	**3.5**	3.5	7.9	184.2
introsort	6.4	3.7	4.9	5.7	4.0	6.0	7.0	6.7
radix_sort	**5.1**	7.8	8.1	**5.2**	7.7	5.0	6.9	8.1

castigator pe test timeout (>60s)

Rezultate big

Doar O(n log n) si radix. Bubble / gnome / insertion sariti. TIMEOUT > 60s.

algoritm	random	sortat	sortat_invers	few_unique	almost_sorted	all_equal	neg_poz	sawtooth
patiencesort	TIMEOUT	TIMEOUT	273.2	TIMEOUT	TIMEOUT	TIMEOUT	TIMEOUT	TIMEOUT
heap_sort	7949	1434	1505	1737	1917	**62.5**	7811	1564
merge_sort	2877	**97.9**	498.3	1256	549.7	80.7	2875	**280.9**
quicksort	2818	605.0	443.0	889.7	437.5	474.7	2720	TIMEOUT
introsort	2164	197.2	**189.3**	653.5	**287.4**	241.7	2137	1780
radix_sort	**1034**	1954	1499	**332.3**	2024	1513	**1331**	1249

castigator pe test timeout (>60s)

Castigatorii, per test

1

radix_sort

5 castiguri

2

merge_sort

4 castiguri

3

insertionsort

2 castiguri

Restul plutonului

Top per coloana

s	random	**radix_sort**	b	random	**radix_sort**
s	sortat	**insertionsort**	b	sortat	**merge_sort**
s	sortat_invers	**merge_sort**	b	sortat_invers	**introsort**
s	few_unique	**radix_sort**	b	few_unique	**radix_sort**
s	almost_sorted	**quicksort**	b	almost_sorted	**introsort**
s	all_equal	**insertionsort**	b	all_equal	**heap_sort**
s	neg_poz	**heap_sort**	b	neg_poz	**radix_sort**
s	sawtooth	**merge_sort**	b	sawtooth	**merge_sort**

Patience a iesit din ring

Pe small random: TIMEOUT.

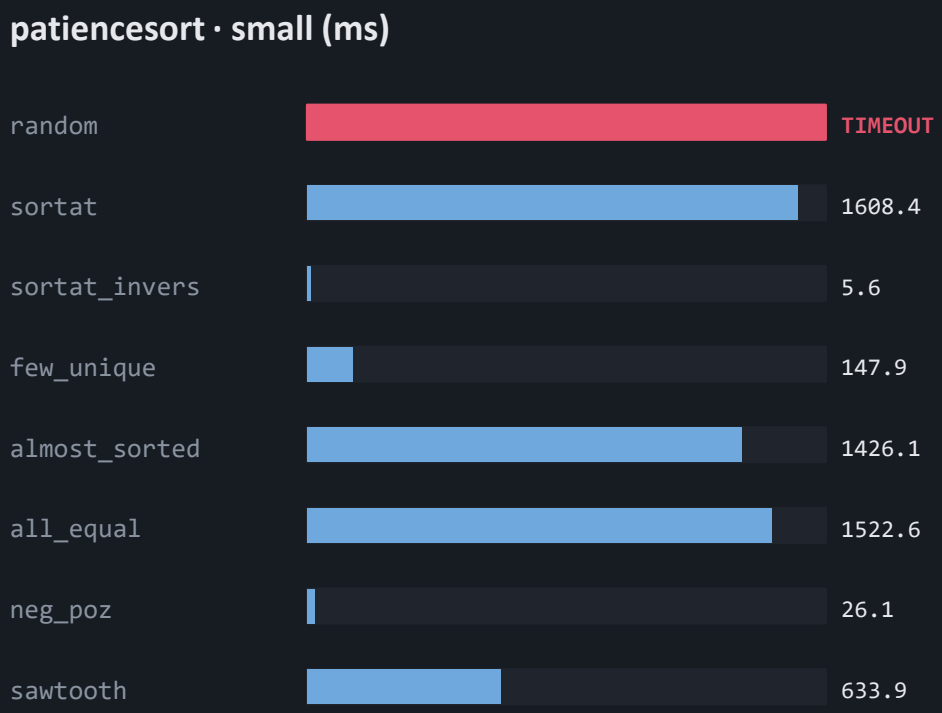
La 20.000 de elemente random, patiencesort nu termina in 60 de secunde. Pe big, picat pe 7 din 8 teste.

De ce

Cautare liniara prin pile la fiecare element, plus un merge final $O(p^2)$. Pe random ies multe pile si totul devine $O(n^2)$ cu constanta urata.

Unde merge totusi

small_sortat_invers: 5.6 ms. Aproape de merge_sort (4.5 ms), pentru ca descendentul iese intr-o singura pila.



Quicksort + sawtooth = capcana pivotului

`big_sawtooth · quicksort`

TIMEOUT. Pe small_sawtooth: **184.2 ms**. De ~30x mai rau decat introsort (6.7 ms).

Problema

Pivotul este v[(begin+end)/2]. Pe sawtooth (1..n/2, n/2..1), mijlocul cade fix pe valoarea cea mai mare. Partitia devine total dezechilibrata, recursie aproape liniara, stack depth O(n).

Diferenta

Introsort foloseste median-of-three si comuta pe heapsort la adancime $2 \cdot \log_2(n)$. Pe acelasi big_sawtooth: 1779.8 ms vs TIMEOUT la quicksort.

Concluzie

Quicksort cu pivot fix nu e safe pentru date adverse. Median-of-three nu e doar microoptimizare, e un safety net. Introsort > quicksort gol in practica.

Cand folosesti ce

n mare, fara presupuneri

→ introsort

robust pe orice input, niciodata catastrofic

n mare, intregi cu chei mici

→ radix_sort

liniar pe big_random (1.0s) si big_few_unique (0.3s)

n mic + stii ca e aproape sortat

→ insertionsort

3 ms pe small_sortat, simplu si cache-friendly

n mare, stii ca e sortat sau aproape

→ merge_sort

skip-ul lui merge prinde rapid 97.9 ms

demonstratii / teorie

→ patience

frumos pe hartie, lent in practica

interviuri

→ quicksort

stii sa-l scrii, dar mentioneaza pivot-ul